Security Review

Cantina Managed

SKY: OFT ADAPTER V2

Cantina Managed review by:
Christoph Michel, Lead Security Researcher
M4rio.eth, Lead Security Researcher

CANTINA

September 4, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Findings **4**
- 3.1 Informational . . . 4
  - 3.1.1 Avoid LayerZero `uint32.max` / `READ_CHANNEL_1` collision for sentinel ID . . . 4
  - 3.1.2 Return zero quote limits for `SENTINEL_EID` . . . 4
  - 3.1.3 Align `oapp-evm-upgradeable` package version with other LayerZero dependencies 5
  - 3.1.4 `ISkyOFTAdapter` / `ISkyRateLimiter` omit the rate-limit and fee view . . . 6
  - 3.1.5 Minor issues, Typos & Documentation . . . 6

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Aug 13th to Aug 16th the Cantina team conducted a review of sky-oapp-oft on commit hash a06a31e3. The team identified a total of **5** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	5	2	3
Total	**5**	**2**	**3**

The Cantina Managed team reviewed Sky's sky-oapp-oft holistically on commit hash b76de539 and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Informational

3.1.1 Avoid LayerZero `uint32.max` / `READ_CHANNEL_1` collision for sentinel ID

Severity: Informational.

Context: SkyOFTAdapter.sol#L29.

Description: LayerZero already uses `type(uint32).max` EID as `READ_CHANNEL_1`. It is not a chain EID, but a virtual route for read-only cross-chain reads.

For example, an Ethereum OApp can request an Avalanche token balance by sending a packet with `dstEid = READ_CHANNEL_1`. The encoded read command separately contains Avalanche's real EID (`30106`), the token address, `balanceOf(vault)` calldata, and the desired block/timestamp and confirmations. No transaction executes on Avalanche - read workers query its state, Read DVNs attest the result, and the verified response is delivered back to the Ethereum OApp.

```
ClientOApp.requestAvalancheBalance(...)
├─ ReadCmdCodecV1.encode(...)
│   // targetEid = 30106
│   // target = avalancheToken
│   // callData = balanceOf(vault)
└─ OAppSender._lzSend(READ_CHANNEL_1, readCommand, ...)
    └─ EndpointV2.send(...)
        └─ MessageLibManager.getSendLibrary(oapp, READ_CHANNEL_1)
            └─ ReadLib1002.send(...)
```

While `EndpointV2` does not special-case `uint32.max`, Ethereum mainnet configures `READ_CHANNEL_1` (`uint32.max = 4294967295`) to use `ReadLib1002` as its default send library. This can be verified with:

```
cast call \
  0x1a44076050125825900e736c501f859c50fE728c \
  'defaultSendLibrary(uint32)(address)' \
  4294967295 \
  --rpc-url "$ETH_RPC_URL"

# 0x74F55Bc2a79A27A0bF1D1A35dB5d0Fc36b9FDB9D
```

Therefore, Sky's global-rate-limit sentinel collides with an identifier already used in LayerZero's public namespace. Note that this isn't of practical impact here, as the `SENTINEL_ID` never reaches the LZ endpoint for the Sky OFT (the functions revert when the peer for that `eid` is missing).

In addition, LZ also sometimes performs `mod 30_000` to normalize the eids (since V2). And `uint32.max mod 30_000 = 17_295`, which doesn't currently map to any chain eid, but this value isn't anything special/reserved either.

Recommendation: As the `SENTINEL_EID` does currently not reach the LZ endpoint, no change is required. Consider keeping this in mind or documenting it in case it reaches the LZ endpoint in the future.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 Return zero quote limits for `SENTINEL_EID`

Severity: Informational.

Context: SkyOFTCore.sol#L124.

Description: The observed behaviors when calling the OFT adapter functions with the `SENTINEL_EID` as the destination `eid` are:

- `SkyOFTCore.quoteOFT(...)` succeeds despite no peer. It computes local fee and rate-limit information (twice from the global rate limit).
- `OFTCoreUpgradeable.quoteSend(...)` reverts with `NoPeer(eid)` in `OAppCoreUpgradeable._getPeerOrRevert(...)`.
- `OFTCoreUpgradeable.send(...)` first debits the user, then reverts with NoPeer(4294967295) when _lzSend(...) resolves the peer. The entire transaction rolls back, including the token transfer and rate-limit updates.

While `quoteOFT` should not revert for arbitrary EIDs, returning `(min, max) = (0, currentGlobalCap)` for `SENTINEL_EID` incorrectly suggests that a transferable amount is available even though sends to this EID cannot succeed.

Recommendation: Special-case `SENTINEL_EID` in `quoteOFT` and return limits of `(min, max) = (0, 0)`.

Sky: Acknowledged.

Cantina Managed: Acknowledged.

3.1.3 Align `oapp-evm-upgradeable` package version with other LayerZero dependencies

Severity: Informational.

Context: package.json#L52.

Description: The currently used LayerZero package versions are:

Package	Version
`oapp-evm`	`0.3.2`
`oapp-evm-upgradeable`	`0.1.3`
`oft-evm`	`3.2.1`
`oft-evm-upgradeable`	`3.2.1`
LayerZero protocol libraries	`3.0.139`

The out-of-line package is `oapp-evm-upgradeable` `0.1.3`. It belongs to a newer dependency generation intended for `oapp-evm` `0.4.1` and LayerZero protocol libraries `^3.0.148`, while the pnpm lock file uses `3.0.139`. Everything else is from the older generation.

The LayerZero code used here did not change between `0.1.2` and `0.1.3`, so this does not currently result in a code-level issue. However, the declared dependency versions should be aligned.

- upgradeable 0.1.2.
- upgradeable 0.1.3.

```
# verified by
npm diff --diff='@layerzerolabs/oapp-evm-upgradeable@0.1.2'
↪  --diff='@layerzerolabs/oapp-evm-upgradeable@0.1.3' -- contracts/
```

Recommendation:: Downgrade `oapp-evm-upgradeable` to `0.1.2`:

```
- "@layerzerolabs/oapp-evm-upgradeable": "0.1.3"
+ "@layerzerolabs/oapp-evm-upgradeable": "0.1.2"
```

Keep the other versions unchanged.

Sky: Fixed in commit b76de539.

Cantina Managed: Fix verified.

3.1.4 `ISkyOFTAdapter` / `ISkyRateLimiter` omit the rate-limit and fee view

Severity: Informational.

Context: ISkyOFTAdapter.sol#L4.

Description: `SkyRateLimiter` has public getters for the accounting mode and the raw per-eid buckets:

```
function rateLimitAccountingType() external view returns (RateLimitAccountingType);
function outboundRateLimits(uint32 _dstEid) external view returns (RateLimit
↪  memory);
function inboundRateLimits(uint32 _srcEid) external view returns (RateLimit memory);
```

`ISkyRateLimiter` only declares `getAmountCanBeSent` / `getAmountCanBeReceived`. The three getters above are implementation-only. `ISkyOFT` has the setters (`setRateLimits`, `setRateLimitAccountingType`) and `feeBalance()`, but not the matching reads.

`ISkyOFTAdapter` only exposes `migrateLockedTokens`. It does not inherit `ISkyOFT` or `ISkyRateLimiter`, so the interface cannot see:

- `SENTINEL_EID` (the lockbox-only global-cap key, SkyOFTAdapter.sol#L29).
- `rateLimitAccountingType()`
- `outboundRateLimits(eid)` / `inboundRateLimits(eid)`.
- The sentinel-aware `getAmountCanBeSent` / `getAmountCanBeReceived` overrides.
- `feeBalance()` (declared on `ISkyOFT`, stored separately on the adapter).

Recommendation: Add the missing getters to `ISkyRateLimiter`.

Sky: Fixed in commit b76de539.

Cantina Managed: Fix verified.

3.1.5 Minor issues, Typos & Documentation

Severity: Informational.

Context: *See each case below.*

Description:

1. OFT_V2_NOTES.md: `SENTINEL_EID` operator rule missing from NatSpec and `OFT_V2_NOTES.md`, and `OFT_V2_NOTES.md` should not live under contracts/.

Recommendation: Consider fixing the above issues.

Sky: Acknowledged.

Cantina Managed: Acknowledged.